

## INFORME TÉCNICO - SOLUCIÓN BI PARA PROCESOS DE INVERSIÓN EN CREDICORP CAPITAL BASADOS EN INDICADORES FINANCIEROS

### 1. Arquitectura Cloud definida

La arquitectura Cloud definida responde a la necesidad de implementar un sistema de inteligencia de negocios orientado a procesamiento batch de datos financieros, con ejecución periódica, bajo un enfoque desacoplado que permita separar claramente las etapas de extracción, almacenamiento, transformación y visualización.

Este diseño busca garantizar escalabilidad bajo demanda, reducción de costos operativos y consistencia en el cálculo de indicadores financieros utilizados para la toma de decisiones de inversión.

A continuación, se realiza una comparación técnica entre Microsoft Azure, Amazon Web Services (AWS) y Google Cloud Platform (GCP) con el objetivo de justificar la selección de Azure como plataforma cloud para la arquitectura implementada, considerando criterios directamente relacionados con los requerimientos técnicos del proyecto.

| Características      | Azure                                                                                                                                                                                                      | AWS                                                                                                                  | GCP                                                                                                     | Ganador                                                                      |
|----------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>Seguridad</b>     | Muy seguro y fácil de usar gracias a Microsoft Entra ID (mismas cuentas corporativas que Power BI). Tiene un lugar seguro para claves (Key Vault) y permite que los servicios se conecten sin contraseñas. | Muy seguro y con mucho control, pero requiere más configuraciones para que todo funcione con herramientas Microsoft. | Seguro y moderno, pero no se integra tan naturalmente con Power BI o SQL Server.                        | Empate: Azure/AWS - Azure gana por tener integración directa con PowerBI/SQL |
| <b>Escalabilidad</b> | Escala automáticamente y puede “apagarse por completo” cuando no se usa, ahorrando dinero. Ideal para procesos ocasionales como el tuyo.                                                                   | Escala muy bien, pero requiere más configuración y no siempre se puede apagar totalmente.                            | Escala rápido, pero está pensado para apps web; para trabajos programados puede tener arranques lentos. | Azure (más simple + ahorro real).                                            |
| <b>Costo/Pricing</b> | Muy económico para trabajos por lotes (batch). No pagas cuando tu contenedor no corre. Azure SQL tiene planes accesibles (S1/S2).                                                                          | Generalmente es más costoso para una arquitectura similar; contenedores y base de datos tienden a cobrar             | Competitivo, pero Cloud SQL y conexiones externas pueden elevar los costos.                             | Azure                                                                        |

|                                              |                                                                                                                        |                                                                                                     |                                                                                                     |       |
|----------------------------------------------|------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------|-------|
|                                              |                                                                                                                        | más.                                                                                                |                                                                                                     |       |
| <b>Ecosistema BI</b>                         | El mejor para Power BI: todo conecta sin configuraciones extra (Azure SQL, Entra ID, refrescos automáticos).           | Tiene Amazon QuickSight, pero Power BI no es nativo y requiere pasos adicionales.                   | Muy fuerte en Looker y BigQuery, pero Power BI no es su enfoque.                                    | Azure |
| <b>Integración con Power BI</b>              | Conexión directa, rápida y segura. No necesitas puertas de enlace ni drivers especiales.                               | Requiere instalar conectores o gateways; más pasos, más mantenimiento.                              | Similar a AWS: requiere drivers y configuraciones adicionales.                                      | Azure |
| <b>Facilidad para desplegar arquitectura</b> | Muy sencillo: Azure Container Apps + Blob + Functions + SQL trabajan juntos sin fricción. Se arma con pocos servicios. | Funciona, pero exige configurar más piezas (Fargate, S3, Notifications, Lambda, RDS). Más complejo. | Funciona con Cloud Run + Cloud Storage + Cloud Functions + Cloud SQL, pero Power BI añade fricción. | Azure |

La comparación se centra en criterios técnicos relevantes para el proyecto, tales como la integración con herramientas de Business Intelligence, la facilidad de implementación de procesos ETL serverless, la compatibilidad con modelos relacionales para el cálculo de KPIs financieros y la reducción de complejidad operativa.

Su integración nativa con Power BI, Azure SQL Database y Microsoft Entra ID reduce significativamente la complejidad operativa y elimina la necesidad de configuraciones adicionales o conectores externos, a ello se suma su modelo de ejecución serverless, que permite escalar bajo demanda y optimizar costos en cargas periódicas como las que requiere este proyecto.

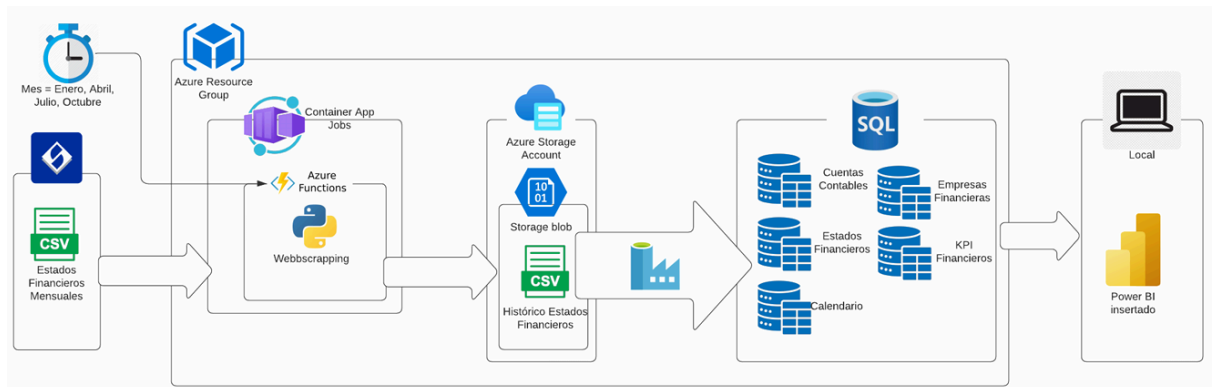
Desde una perspectiva de arquitectura cloud, la elección de Azure permite reducir la complejidad del diseño, centralizar la gestión de seguridad y asegurar una integración coherente entre los servicios de datos, procesamiento y visualización, lo que resulta crítico para la estabilidad y mantenibilidad del sistema de inteligencia de negocios implementado.

## 2. Servicios utilizados

Para garantizar un flujo de Inteligencia de Negocios eficiente, automatizado y escalable, se diseñó una arquitectura basada en servicios cloud administrados de Microsoft Azure, esta arquitectura integra todos los componentes necesarios para ejecutar el proceso ETL de forma programada, almacenar los datos en un repositorio

seguro, transformarlos automáticamente y habilitarlos para su análisis mediante Power BI.

La selección de servicios administrados y serverless se realizó con el objetivo de evitar la gestión de infraestructura, reducir dependencias operativas y asegurar una ejecución reproducible del flujo ETL.



a. Azure Container Apps – Fase: ETL (Extracción)

Se utiliza para ejecutar el script Python encargado de obtener los datos desde la fuente externa. Este servicio permite ejecutar contenedores de forma serverless, lo que significa que no requiere servidores activos ni infraestructura permanente.

Funciones principales:

- Ejecutar el scraping o descarga del dataset según una programación (por ejemplo, trimestral).
- Generar el archivo CSV procesado por Python.
- Enviar automáticamente el archivo generado hacia Azure Blob Storage.

Ventajas:

- Escala desde cero y no genera costos cuando no se ejecuta
- Permite usar librerías pesadas (pandas, requests, scraping)
- Ideal para procesos batch programados.

b. Azure Blob Storage – Fase: Almacenamiento de archivos

Cumple la función de repositorio central dentro del Data Lake, donde los archivos descargados por los contenedores se almacenan inicialmente en formato original (raw) y posteriormente Blob Storage también almacena los datasets ya transformados (curated), sirviendo como puente entre la ingesta y la etapa de procesamiento.

Funciones:

- Recibir el archivo CSV recién generado por el contenedor.
- Mantener un historial de cargas por versión o fecha.
- Servir como punto de origen para el proceso de carga hacia la base de datos.

Este servicio asegura alta disponibilidad, versionamiento y costos muy bajos para grandes volúmenes de datos.

c. Azure Functions – Fase: ETL (Transformación y carga)

Ejecuta la lógica de transformación de datos bajo un modelo totalmente serverless, donde sus principales funciones son:

- Leer el archivo desde Azure Blob Storage.
  - Validar columnas, estandarizar formatos y aplicar reglas de limpieza.
  - Preparar los datos transformados para su posterior carga estructurada.
  - Depositar los datos transformados en la zona correspondiente del Data Lake.
- d. Azure Data Factory – Fase: ETL (Carga estructurada)
- Azure Data Factory es el servicio responsable de la orquestación y ejecución de la carga estructurada de datos hacia Azure SQL Database, este servicio consume los datasets almacenados en Azure Blob Storage y ejecuta los procesos de inserción de datos de forma controlada, reproducible y auditable. Funciones principales:
- Orquestar el flujo de carga desde el Data Lake hacia Azure SQL Database.
  - Ejecutar procesos ETL de forma programada.
  - Gestionar errores, reintentos y monitoreo de ejecuciones.
  - Garantizar la consistencia y trazabilidad de los datos cargados.
- e. Azure SQL Database – Fase: DataMart
- Almacena la información ya procesada y lista para consulta, es el motor relacional donde residen las tablas que alimentarán directamente los dashboards. Funciones:
- Guardar los datos finales provenientes del proceso ETL.
  - Exponer información limpia y estructurada para herramientas analíticas.
  - Ejecutar consultas requeridas por Power BI.
  - Permitir el cálculo de KPIs mediante vistas o procedimientos almacenados.
- f. Power BI – Fase: Visualización y Análisis
- Se conecta a Azure SQL Database para consumir los datos procesados y generar las visualizaciones necesarias para el análisis de indicadores financieros. Funciones:
- Leer las tablas y vistas alojadas en SQL Database.
  - Construir dashboards interactivos.
  - Permitir análisis comparativos por entidad, periodo o indicador.
  - Dar soporte a la toma de decisiones mediante KPIs claros y actualizados.

### 3. Diseño del Data Lake (bronce-plata-oro)

El Data Lake diseñado tiene el objetivo de proveer una estructura ideal para la ingesta, el almacenamiento, procesamiento y disponibilización de la información financiera de los bancos del sistema financiero del Perú junto con sus respectivos indicadores financieros.

Para así cubrir las necesidades de robustez, trazabilidad y escalamiento de Credicorp, se utilizó la arquitectura bronce-plata-oro para el diseño de la solución, el cuál es la siguiente:

1. Capa de Bronce:

En esta capa se conservan los archivos con la información de los estados financieros de las empresas producto del proceso de webscrapping a la página web de la SBS. Estos archivos no han sido modificados y permiten mantener un histórico de los archivos descargados

Contenido: Archivos Excel (.xls)

Justificación: Permite la trazabilidad y auditoría de los datos, dado que permite identificar el origen de los datos

2. Capa de Plata:

En esta capa se encuentra la información financiera ya procesada y estructurada en dataset con la información necesaria para ser importada en una bases de datos

Procesos Aplicados:

- Eliminación de letras con tilde (á, é, í, ó, ú)
- Eliminación de caracteres extraños: \*, +, %, ., (, ), etc
- Reemplazo de los espacios ( ) por guiones bajos (\_)
- Espacios en blanco rellenos con: 0
- Estandarización del formato del formato de fecha y valores numéricos.
- Estructuración de los datos especificando empresa, fecha y añadiendo datos: tipo de cuenta y nivel de la cuenta.

Diseño Lógico:

- Código de cuenta contable
- Nombre de la cuenta
- Valor de la cuenta
- Tipo de la cuenta
- Tipo de estado financiero al que pertenece
- Empresa
- Periodo contable

Contenido: Archivo .csv

Justificación: Preparación de los datos contables para ser importados a una base de datos.

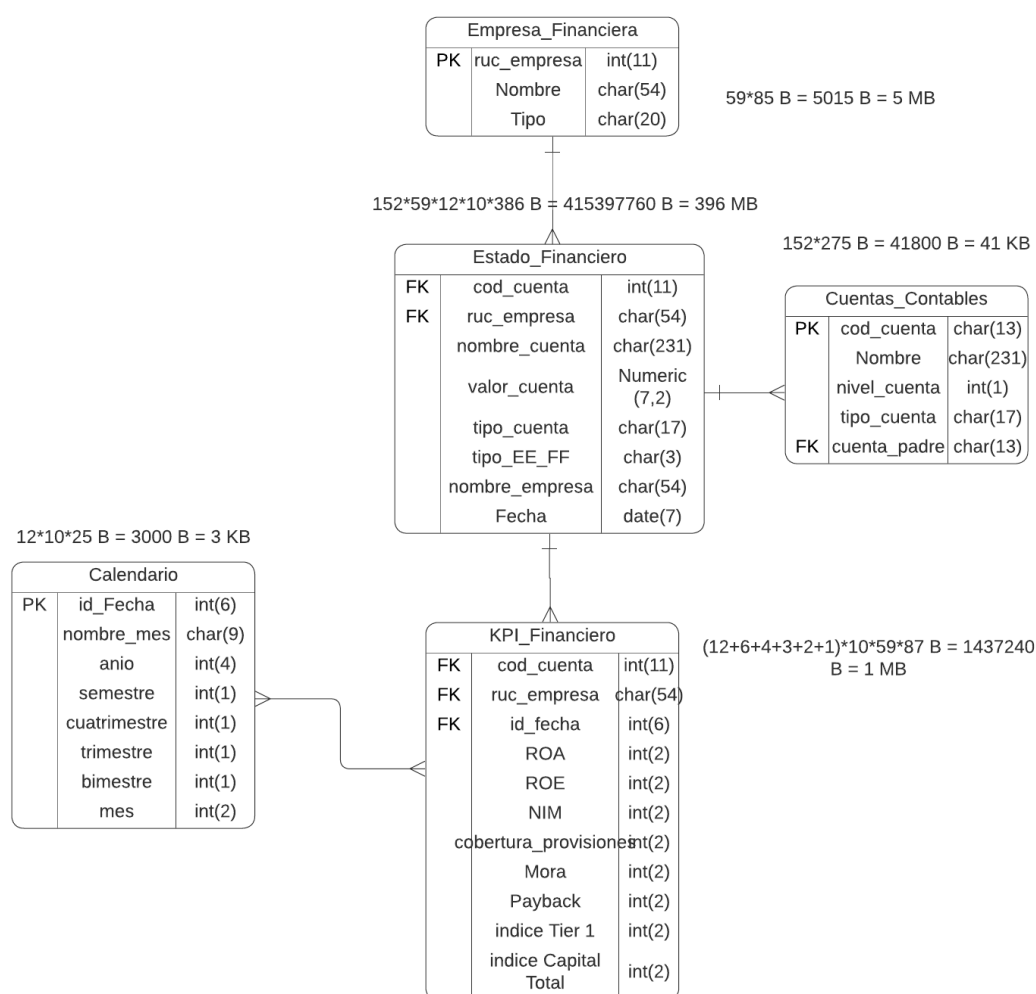
3. Capa de Oro:

En esta capa se encuentra la información financiera ya subida a una base de datos y disponible para su uso al igual que los indicadores financieros o KPI's financieros ya calculados.

Contenido: Tabla SQL de datos financieros (tabla\_cuentas\_contables) y de indicadores (KPI's financieros).

#### 4. Modelo Dimensional Implementado

El modelo dimensional está diseñado siguiendo el siguiente esquema para la base de datos:



Entre los componentes de la base de datos tenemos:

- La Tabla de Empresa\_Financiera se estima que cada fila ocupará un espacio de almacenamiento de 85 B, y considerando que en total se almacenarán datos de 59 empresas financieras, el total de total de almacenamiento requerido será de 5015 B <> 5 KB. Ya que se tratan de empresas del mundo real, nos basaremos en el principio de perpetuidad de la empresa para considerar que este valor (5 KB) no cambiará.

- La tabla de Cuentas\_Contables, se estima que cada fila ocupará un espacio de almacenamiento de 275 B, y considerando que en total se almacenarán datos de 152 cuentas contables, el total de total de almacenamiento requerido será de 41800 B <> 41 KB. Debido a que son cuentas contables establecidas como necesarias por la SBS, se considerará que este valor (41 KB) no variará.
- La tabla Calendario, se estima que cada fila ocupará un espacio de almacenamiento de 25 B, y considerando que en un inicio se almacenarán datos de todos los meses de los últimos 10 años (2015-2025) se almacenarán datos de 120 meses calendarios, por lo que, el total de espacio de almacenamiento requerido será de 3000 B <> 3 KB. Debido a que el objetivo del software es mantener actualizados los datos financieros, este valor si variará a una razón de 300 B por año o 25 B por mes.
- La tabla Estado\_Financiero, se estima que, en un caso extremo, cada fila ocupará un espacio de almacenamiento de 386 B, y considerando que en un inicio se almacenarán datos de cada una de las cuentas contables (152) de todas las empresas financieras (59) de los meses (12) de los últimos 10 años (2015-2025) se almacenarán un total de 1076160 datos, por lo que, el total de total de espacio de almacenamiento requerido será de 415397760 B <> 396 MB. Debido a que el objetivo del software es mantener actualizados los datos financieros, este valor si variará a una razón de 107616 B <> 0.1 MB por año o 8.8 KB por mes.
- La tabla KPI\_Financiero, se estima que, en un caso extremo, cada fila ocupará un espacio de almacenamiento de 46 B, y considerando que en un inicio se almacenarán datos de cada una de los ratios financieros de todas las empresas financieras (59) tanto anual, mensual (12), semestral (2), cuatrimestral (3), trimestral (4) y bimestral (6) de los últimos 10 años (2015-2025), en total se almacenarán 16520 datos, por lo que, el total de total de espacio de almacenamiento requerido será de 759920 B <> 1 MB. Debido a que el objetivo del software es mantener actualizados los ratios financieros, este valor si variará a una razón de 75992 B <> 0.1 MB por año o 8.8 KB por mes.

Teniendo en cuenta lo anteriormente expuesto, dado que es una base de datos con crecimiento dinámico para que este modelo sea implementado debe tener en cuenta sus expectativas de crecimiento que sigue la siguiente fórmula:

Espacio Requerido: 42 MB \* (Años almacenados + 1)

## 5. Los procesos ETL aplicados

**Proceso de Extracción (Extract):**

El proceso de extracción consiste en realizar webscrapping de la página web de la SBS en la cual se encuentran archivos excel (.xls), una vez extraído el archivo es almacenado en azure blob storage (/rawdata), en la carpeta raw data.

Justificación: Permite la trazabilidad y la depuración de errores luego del proceso de transformación de los datos, además de permitir la auditoría de los datos y del proceso.

**Proceso de Transformación (transform):**

El proceso de transformación consiste en:

- La eliminación de elementos o símbolos extraños
- La eliminación de espacios vacíos
- Eliminación de tildes
- Reemplazo de espacios por guiones bajos
- Reestructuración de los datos para ser subidos a la base de datos, específicamente a la tabla estados\_financieros

Para ello se realizó una consulta a la tabla cuentas\_contables y se fabricó un “molde” o “plantilla” con el mismo orden de las cuentas contables del archivo excel para posteriormente agregarle los datos del valor de la cuenta, empresa, estado financiero y fecha.

Posteriormente se genera un archivo csv con los datos ya estructurados y consolidados y se almacenan en Azure Blob Storage (/curateddata)

Todo esto fue desarrollado con un código python.

Justificación: Se decidió no cargar los datos directamente a la base de datos debido a que este método permite facilitar el proceso de corroboración de la correcta transformación de los datos, ya que evita la necesidad de utilizar consultas sql. Simplemente consistiría en revisar el excel correspondiente y contrastar los valores

**Proceso de Carga (Load):**

Para el proceso de carga se desarrolló un pipeline utilizando Azure Data Factory, el cual consiste en un script que importa el archivo presente en Azure Blob Storage (/curateddata) a la tabla estados\_financieros de Azure SQL Server.

Justificación: Automatiza el proceso de carga de datos sin necesidad de complejizar el código utilizado.

**6. Scripts utilizados y consultas SQL**



Se optó por el uso de SQL nativo en Azure SQL Database debido a su capacidad de procesamiento set-based, su compatibilidad con procedimientos almacenados y su integración directa con herramientas de visualización como Power BI, reduciendo la latencia entre el procesamiento y el consumo analítico.

#### 6.1. Creación de tablas en la base de datos

Se definieron las siguientes tablas:

##### **Tabla Base: [estados\\_financieros](#)**

Esta tabla almacena los datos financieros tal como vienen del archivo CSV almacenado en el blob storage.

Justificación técnica:

- Permite volver a procesar la información si cambian los cálculos.
- Sirve como base para generar todas las demás tablas.

##### **Tablas dimensionales**

##### **[cuentas\\_contables](#)**

Esta tabla organiza las cuentas financieras en una estructura jerárquica.

Justificación técnica:

- Permite representar la estructura real del plan contable financiero.
- Facilita futuras extensiones del modelo sin modificar la tabla de hechos.
- Habilita análisis por niveles contables (cuenta mayor, subcuenta, etc.).

##### **[empresa\\_financiera](#)**

Esta tabla contiene el listado de empresas analizadas.

Justificación técnica:

- Normaliza la información de empresas.
- Evita redundancia en la tabla de hechos.
- Facilita el filtrado por empresa en los reportes.

##### **[calendario](#)**

Esta tabla organiza la información relacionada al tiempo.

Justificación técnica:

- Optimiza consultas temporales evitando cálculos de fechas en cada consulta.
- Permite análisis por año, trimestre, semestre, bimestre y mes.
- Es una buena práctica estándar en modelos dimensionales.

##### **Procedimiento almacenado**

##### **[generar\\_kpis](#)**

El procedimiento almacenado generar\_kpis se encarga de calcular automáticamente todos los indicadores financieros.

Justificación técnica:

- Centraliza los cálculos en un solo lugar.
- Evita errores por cálculos duplicados o inconsistentes.
- Permite recalcular los indicadores cada vez que se cargan nuevos datos.

El orden de ejecución es el siguiente:

- 1) Crear tabla base: estados\_financieros
- 2) Cargar datos CSV en estados\_financieros
- 3) Crear tabla catálogo: cuentas\_contables
- 4) Crear dimensión: empresa\_financiera y poblar
- 5) Crear dimensión: calendario y poblar
- 6) Crear tabla de hechos: kpi\_financiero
- 7) Crear procedimiento: generar\_kpis
- 8) Ejecutar procedimiento: generar\_kpis
- 9) Validar datos en todas las tablas

Nota: todos los scripts de creación y población se pueden encontrar en el siguiente enlace:

[Scripts SQL](#)

## **7. Visualizaciones finales implementadas**

### **7.1 Conexión con Power BI**

Se utilizó Power BI Desktop (ya que tiene una conexión directa con Azure, permitiendo la actualización en tiempo real) como herramienta de visualización, conectándose directamente a Azure SQL Database en modo Import.

Justificación técnica:

- El modo Import ofrece mejor rendimiento en consultas analíticas.
- Reduce la latencia durante la interacción con los dashboards.
- Permite el uso eficiente del modelo dimensional ya procesado.

### **7.2 Modelo de Datos en Power BI**

Las visualizaciones se construyeron sobre un modelo estrella compuesto por:

- kpi\_financiero como tabla de hechos.
- empresa\_financiera y calendario como dimensiones.

Justificación técnica:

- Minimiza la cantidad de relaciones.

- Optimiza el motor de almacenamiento columnar de Power BI.
- Facilita la creación de medidas y filtros.

### 7.3 Tipos de Visualizaciones Utilizadas

#### **Indicadores KPI (ROA, ROE, NIM, Mora, Cobertura de Provisiones)**

Se utilizaron tarjetas KPI para representar valores clave.

Justificación técnica:

- Permiten una lectura inmediata del desempeño financiero.
- Son adecuadas para métricas agregadas y comparables.
- Mejoran el entendimiento del usuario final.

#### **Gráficos Temporales**

Se emplearon gráficos de líneas para analizar la evolución de indicadores financieros a lo largo del tiempo.

Justificación técnica:

- Facilitan la identificación de tendencias y variaciones.
- Permiten comparaciones entre períodos.
- Son ideales para análisis histórico de desempeño financiero.

#### **Filtros y Segmentadores**

Se implementaron segmentadores por empresa y período.

Justificación técnica:

- Permiten análisis dinámico sin recalcular datos.
- Mejoran la experiencia del usuario final.
- Reducen la necesidad de múltiples dashboards estáticos.

Nota. el dashboard final se puede encontrar en el siguiente enlace: [Dashboard Final](#)

### 8. Uso del repositorio GitHub

- Rama principal: main (docente).
- Ramas por grupo: feature/grupo03-init.
- Las integraciones se realizan por Pull Request (PR)